🐳 Guia Essencial: Entendendo o Docker do Zero

🏷️ **Tags:** #docker #devops #programação #tutorial #iniciante

🤔 O que é o Docker?

O **Docker** é uma plataforma que permite baixar, criar, executar e gerenciar aplicações dentro de ambientes isolados chamados **Contêineres** (containers).

> ⓘ 💡 **Para entender melhor**
>
> Imagine que você precisa instalar um programa que depende de várias coisas para funcionar: banco de dados, linguagem de programação, servidor e configurações. Normalmente, você instalaria tudo isso manualmente. O Docker resolve isso embalando tudo o que o programa precisa em um único "pacote" isolado, garantindo que rode perfeitamente em qualquer máquina.

🛠️ Como o Docker facilita a sua vida?

Imagine que você deseja utilizar o banco de dados MySQL para um projeto.

🐌 O jeito antigo (sem Docker)

1. 🌐 Acessar o site do MySQL e baixar o instalador.
2. ⚙️ Executar a instalação e torcer para não dar conflito com nada no seu PC.
3. 🔐 Configurar usuário, senha e o serviço no sistema operacional.

🚀 O jeito Docker

Com apenas **um comando** no terminal, você resolve tudo:

Bash

```
docker run --name meu-mysql -e MYSQL_ROOT_PASSWORD=123456 -d mysql:8
```

(O Docker procura a Imagem do MySQL, baixa da internet se necessário, cria o ambiente isolado, configura a senha e inicia o banco. Tudo pronto em segundos!)

🧩 Conceitos Fundamentais

Esses são os dois pilares do Docker. Para nunca mais esquecer, use a analogia da confeitaria:

🖊📄 Imagem (A Receita)

É o modelo estático. Ela contém as instruções, bibliotecas e arquivos necessários para rodar um programa (ex: a receita escrita de um bolo de chocolate).

📇🍩 Contêiner (O Bolo)

É a aplicação viva e rodando, criada a partir da imagem. Você pode usar a mesma "receita" (Imagem) para criar quantos "bolos" (Contêineres) quiser, e um não interfere no outro.

📋🌍 Onde encontro essas Imagens?

No **Docker Hub**, o repositório público do Docker. É como se fosse a "loja de aplicativos" oficial. Lá você encontra imagens prontas do Ubuntu, Node.js, Python, PostgreSQL, entre milhares de outras.

💻 Preparando o Terreno: Instalação

□ No Windows (via Docker Desktop)

O Docker Desktop oferece uma interface gráfica amigável e utiliza o WSL 2 (Windows Subsystem for Linux) nos bastidores.

1. **Ative o WSL 2:** Abra o PowerShell como *Administrador* e digite:

   PowerShell

   ```
   wsl --install
   ```

⚠ Atenção

Reinicie o seu computador após a instalação do WSL.

2. **Baixe o Docker:** Acesse o site do Docker Desktop e baixe a versão para Windows.
3. **Instale:** Execute o arquivo baixado, certifique-se de marcar a opção de usar o **WSL 2** e avance até concluir.
4. **Inicie:** Abra o Docker Desktop pelo menu Iniciar e aguarde o motor carregar 🐳.

🐧 No Linux (Ubuntu/Debian via Terminal)

1. **Atualize o sistema:**

Bash

```
sudo apt update && sudo apt install ca-certificates curl
```

2. **Adicione a chave oficial de segurança:**

Bash

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

3. **Adicione o repositório e instale:**

Bash

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo
"${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

✓ ✅ **Teste a sua instalação**

Abra o seu terminal (Windows ou Linux) e digite:

`docker run hello-world`

Se uma mensagem amigável aparecer, o Docker está funcionando perfeitamente! 🎉

🎯 Mão na Massa: Comandos Essenciais

Vamos subir um servidor web real (Nginx) para testar.

1️⃣ Baixando uma imagem (opcional)

Bash

```
docker pull nginx
```

2️⃣ Executando o contêiner

Bash

```
docker run -d -p 8080:80 --name meu-site nginx
```

> ⓘ 🔍 **Entendendo a sopa de letrinhas:**
>
> - `-d` : Roda em segundo plano (*detached*), liberando seu terminal.
> - `-p 8080:80` : Liga a porta `8080` do seu PC à porta `80` do contêiner.
> - `--name meu-site` : Dá um nome fácil de lembrar.
> - `nginx` : O nome da imagem utilizada.

🌐 *Abra seu navegador e acesse* `http://localhost:8080`*. Você verá a página inicial do Nginx rodando!*

📄 Cheat Sheet (Resumo de Sobrevivência)

⌨️ Comando	❓ O que faz?
`docker pull <imagem>`	📥 Baixa a imagem do Docker Hub para o seu PC.
`docker run <imagem>`	🚀 Cria e executa um contêiner a partir de uma imagem.
`docker images`	🖼️ Lista todas as imagens salvas localmente.
`docker ps`	🏃 Lista os contêineres que estão rodando no momento.
`docker ps -a`	🛑 Lista **todos** os contêineres (rodando e parados).
`docker stop <nome>`	⏸️ Para um contêiner em execução.
`docker start <nome>`	▶️ Inicia um contêiner que estava parado.
`docker rm <nome>`	🗑️ Exclui um contêiner (ele precisa estar parado primeiro).
`docker rmi <imagem>`	💥 Exclui uma imagem do seu computador.

> 🔥 💥 **Dica de Ouro**
>
> Você não precisa dar `docker pull` antes de `docker run`. Se você rodar um contêiner de uma imagem que ainda não tem, o Docker é esperto o suficiente para baixá-la automaticamente primeiro!

> ⓘ 📚 **Documentação Oficial**

Caso não consiga baixar as imagens ou precise de informações mais detalhadas, acesse a **Documentação Oficial do Docker**.